National University of Computer & Emerging Sciences
Chiniot – Faisalabad Campus

ARTIFICIAL INTELLIGENCE | TERM PROJECT REPORT

Dynamic Pathfinding Agent with Real-Time Replanning

A grid-based AI system implementing A and Greedy Best-First Search with dynamic obstacle detection and instant path replanning*

Author	Zoha Ashraf
Roll No.	24F-3019
Course	Artificial Intelligence
Institution	NUCES — Chiniot-Faisalabad Campus
GitHub	github.com/ZohaAshraf/Dynamic-Pathfinding-Agent
LinkedIn	linkedin.com/in/zohashraf/
Medium	medium.com/@f243019

Table of Contents

1. Abstract
2. Introduction
3. System Design & Architecture
4. Algorithm Implementation
5. Dynamic Obstacles & Replanning
6. Visualization & Metrics
7. Experimental Results & Screenshots
8. Pros & Cons — Algorithm Comparison
9. Conclusion
10. References

1. Abstract

This report presents a Dynamic Pathfinding Agent — an interactive, grid-based artificial intelligence system developed as part of the Artificial Intelligence course at NUCES Chiniot-Faisalabad Campus. The agent navigates from a designated Start cell to a Goal cell using two informed search algorithms: A* Search and Greedy Best-First Search (GBFS). Both algorithms are supported by two admissible heuristics — Manhattan Distance and Euclidean Distance — selectable at runtime through a professional graphical user interface built with the Pygame library.

The defining feature of this project is its Dynamic Mode, wherein new obstacles are introduced stochastically during agent movement. Upon detecting a blocked path, the agent immediately replans its route from its current position without resetting the simulation. The system visualises every stage of the search process — frontier expansion, visited nodes, the final path, and agent movement — with a real-time metrics dashboard displaying nodes visited, path cost, and execution time in milliseconds.

Keywords: Pathfinding, A* Search, Greedy Best-First Search, Heuristic Functions, Dynamic Replanning, Pygame, Grid Environment, Informed Search.

2. Introduction

Pathfinding is a fundamental problem in Artificial Intelligence that involves finding a navigable route between two points in a structured environment. It underpins a wide range of real-world applications — from GPS navigation systems and robotic motion planning to video-game non-player characters and autonomous vehicle routing. The challenge increases significantly when the environment is not static: obstacles may appear, disappear, or shift while the agent is in motion, requiring the system to adapt intelligently rather than simply re-execute a pre-computed plan.

2.1 Importance of Dynamic Replanning

Classical pathfinding algorithms such as Dijkstra's and basic A* assume a fully observable, unchanging environment. In practice, this assumption is rarely valid. A robot navigating a warehouse must avoid human workers who move unpredictably. A drone must reroute around unexpected air traffic. An autonomous vehicle must respond to sudden roadblocks. Dynamic replanning is therefore not an academic curiosity but a practical necessity.

This project addresses this challenge by extending standard A* and GBFS with an on-demand replanning mechanism that triggers only when the agent's current planned route is verified to be blocked — minimising unnecessary computation and keeping the agent moving efficiently.

2.2 Real-World Applications

- Robotics: mobile robots replanning around dynamic obstacles in factory floors.

- Autonomous Vehicles: real-time rerouting in response to road closures or accidents.
- Video Games: NPC agents adapting to player-placed obstacles or destroyed terrain.
- Logistics: delivery drone path adjustment in response to restricted airspace.
- Emergency Services: routing responders around dynamically-changing incident zones.

3. System Design & Architecture

The application is implemented as a single Python module to maintain simplicity appropriate for academic review. It is structured into the following logical layers:

3.1 Grid Environment

The environment is modelled as a two-dimensional grid of cells. Each cell is either passable (empty) or impassable (a wall/obstacle). The grid is fully configurable: the user enters the number of rows, columns, and obstacle density percentage before the main window opens. The grid supports sizes from 5x5 up to 50x80 cells.

Property	Specification
Grid Type	2-D rectangular cell grid
Movement	4-directional (Up, Down, Left, Right)
Edge Cost	Uniform — 1 per step
Start Node	Fixed top-left corner (row 0, col 0)
Goal Node	Fixed bottom-right corner
Obstacle Source	Random generation OR interactive mouse painting
Min / Max Size	5×5 to 50×80 cells

3.2 Node Class

Each grid cell is represented by a **Node** object storing the following fields:

- **row, col** — grid coordinates of the cell.
- **is_wall** — Boolean flag; True if the cell is an obstacle.
- **g** — exact cost from the start node to this node.
- **h** — heuristic estimate of remaining cost to the goal.
- **f** — evaluation value: f = g + h (A*) or f = h (GBFS).
- **parent** — reference to the predecessor node for path reconstruction.

3.3 Graphical User Interface

The GUI is built with Pygame and divided into two regions: a **grid area** on the left where the search animation is rendered, and a **control panel** on the right housing all buttons, the metrics dashboard, and the colour legend.

Cell Type	Colour	Hex Code
Start Node	Emerald Green	#10b981
Goal Node	Royal Blue	#3b82f6
Obstacle	Dark Charcoal	#1f2937
Frontier	Amber	#f59e0b
Visited	Muted Red	#ef4444
Final Path	Bright Lime	#22c55e
Background	Deep Slate	#0f172a
Agent	White	#ffffff

4. Algorithm Implementation

4.1 A* Search

A* is an informed best-first search algorithm that evaluates each candidate node using a combined cost function:

$$f(n) = g(n) + h(n)$$

where **g(n)** is the exact cost of the best known path from the start to node n, and **h(n)** is an admissible heuristic estimate of the remaining cost from n to the goal. A* maintains a min-heap priority queue (Python's heapq module) ordered by f(n). At each step the node with the lowest f value is expanded; its neighbours are relaxed if a shorter path is discovered.

A* is both **complete** (always finds a path if one exists) and **optimal** (finds the shortest path) when the heuristic is admissible — i.e., it never overestimates the true cost. Both Manhattan and Euclidean distance satisfy this condition on a 4-directional unit-cost grid.

Pseudo-code:

```
OPEN ← {start}; start.g ← 0; start.f ← h(start)
CLOSED ← {}
WHILE OPEN is not empty:
    current ← node in OPEN with lowest f
    IF current == goal: RETURN reconstruct_path(goal)
    move current from OPEN to CLOSED
```

```
    FOR each neighbour of current:
        IF neighbour in CLOSED: skip
        new_g ← current.g + 1
        IF new_g < neighbour.g:
            update neighbour.g, h, f, parent; push to OPEN
RETURN [] (no path found)
```

4.2 Greedy Best-First Search

GBFS simplifies A* by discarding the actual path cost g(n) entirely, relying solely on the heuristic:

$$f(n) = h(n)$$

By always expanding the node that appears closest to the goal according to the heuristic, GBFS typically reaches the goal faster than A* on open, unobstructed grids. However, it sacrifices the optimality guarantee — it can be misled by obstacles and produce paths significantly longer than the true shortest route. It is therefore classified as **complete but not optimal** on finite graphs.

4.3 Heuristic Functions

Heuristic	Formula	Notes
Manhattan	h = |r1 - r2| + |c1 - c2|	Exact lower bound for 4-dir movement. Recommended.
Euclidean	h = sqrt((r1-r2)^2 + (c1-c2)^2)	Slightly underestimates. Admissible. Wider expansion.

Both heuristics are **admissible**: they never overestimate the true minimum cost to the goal. Manhattan distance is the tighter bound for 4-directional grids because it precisely models the minimum number of steps required ignoring obstacles, whereas Euclidean distance takes diagonal shortcuts that are not actually traversable.

5. Dynamic Obstacles & Replanning

5.1 Obstacle Spawning Mechanism

When Dynamic Mode is enabled, the simulation introduces stochastic environmental change. At every agent movement tick, a random number is drawn from a uniform distribution over [0, 1). If this value falls below the configured threshold (default: 0.04, i.e. a 4% probability per step), a new wall is placed on a randomly selected cell that satisfies the following constraints:

- The cell is not already a wall.
- The cell is not the Start node.
- The cell is not the Goal node.

- The cell is not the agent's current position.

This ensures that the agent is never instantly trapped by a single spawn event and that the start and goal positions remain accessible as long as any route exists.

5.2 Blockage Detection

Immediately after each obstacle spawns, the system scans the agent's remaining planned route — from the agent's current index to the end of the path list — checking whether any node has become a wall. This scan runs in O(L) time where L is the remaining path length, executing well within a single animation frame even on large grids.

5.3 Replanning Strategy

If a blockage is detected, the system triggers an immediate replan:

- The grid's start position is temporarily reassigned to the agent's current cell.
- All node search costs (g, h, f, parent) are reset — walls are preserved.
- The selected algorithm (A* or GBFS) is re-invoked from the new start position.
- If a new path is found, the agent continues walking immediately.
- If no path exists, the state transitions to NO_PATH and the simulation halts.
- The original start position is restored after the search completes.

This approach is highly efficient: only lightweight search metadata is reset and recomputed. On a typical 20×30 grid, replanning completes in under 2 milliseconds, making the transition imperceptible to the user. The agent seamlessly continues its journey along the new route without any visible interruption.

6. Visualization & Metrics

6.1 Search Animation

The search visualisation plays back the algorithm's execution frame-by-frame after the computation completes. At each tick one additional frame is rendered, advancing the visible frontier and visited sets forward in time. This separation between computation and animation ensures the GUI never freezes during heavy searches. The animation speed is configurable (default: 25 ms per frame for search, 120 ms per step for agent movement).

6.2 Agent Movement

Once the search animation completes, a white circular agent marker advances along the final path node by node. In Dynamic Mode, obstacle spawning and blockage detection occur at each agent movement step. The path is redrawn in lime green at all times, making the agent's route clearly visible against the

visited (red) and frontier (amber) cells.

6.3 Metrics Dashboard

Metric	Description	Update Timing
Nodes Visited	Count of nodes in the closed set	After search completes
Path Cost	Number of steps from start to goal	After search completes
Execution Time	Wall-clock time of the search in milliseconds	After search completes
Algorithm	Currently selected algorithm (A* / GBFS)	Instant on toggle
Heuristic	Manhattan or Euclidean	Instant on toggle
Dynamic Mode	ON / OFF status	Instant on toggle
Grid Size	Rows × Columns	Set at startup
Status	Current state (IDLE/ANIMATING/WALKING/DONE/NO_PATH)	Each frame

7. Experimental Results & Screenshots

The following screenshots were captured from live runs of the application under different configurations. They illustrate the distinct visual stages of the search process and demonstrate both typical and edge-case behaviour.

7.1 Dense Frontier Expansion (A* on obstacle-heavy grid)

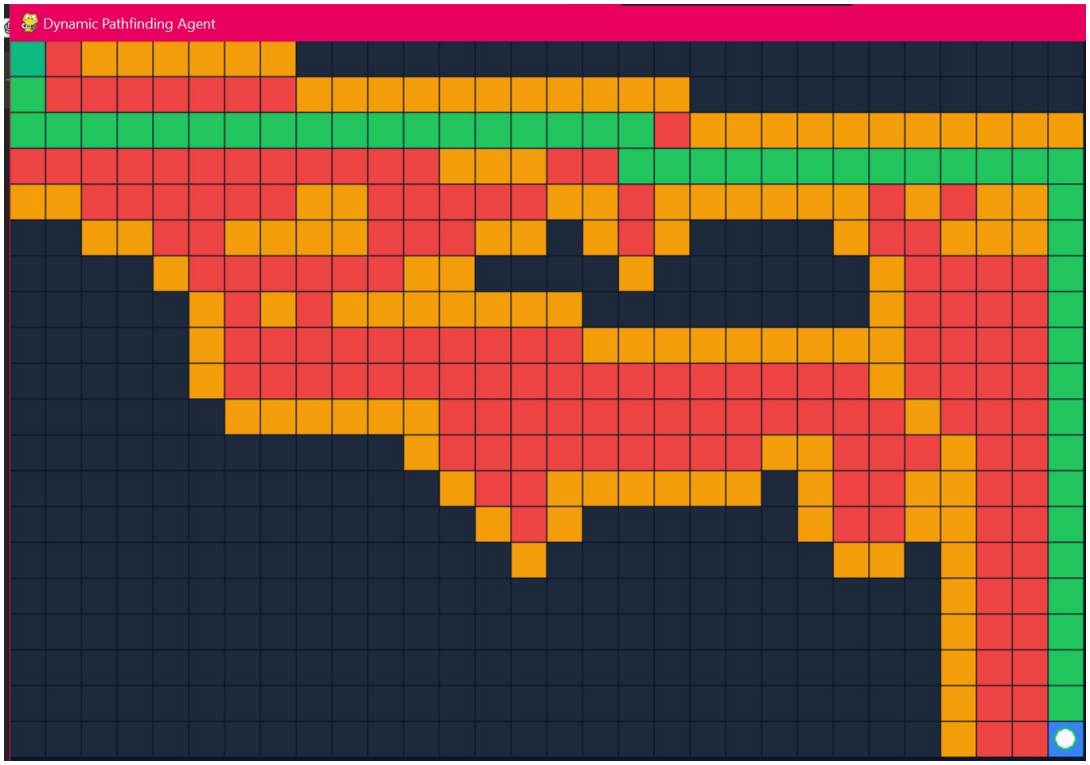

Figure 1: A search on a dense grid. Amber cells show the frontier; red cells are the visited/closed set. The lime path traces from start (top-left) to the agent (white dot, bottom-right area).*

This screenshot demonstrates A* navigating a grid with approximately 30% obstacle density. The wide spread of amber (frontier) and red (visited) cells illustrates how A* systematically explores the search space, balancing actual cost and heuristic estimate. The lime-green path shows the optimal route found. The characteristic 'wavefront' shape of the explored region is a hallmark of A* with Manhattan heuristic.

7.2 Full Grid Coverage — No Obstacles (GBFS)

Figure 2: GBFS on a nearly obstacle-free grid. Almost the entire grid is visited (red) as GBFS explores aggressively. The agent (white dot) walks the lime path while amber cells indicate the remaining frontier.

This screenshot captures GBFS operating on a near-empty grid. Because GBFS uses only the heuristic and ignores actual path cost, it explores most of the grid before converging on a path. While the path found is still functional, the sheer number of visited nodes (shown in red) demonstrates why GBFS is suboptimal in obstacle-sparse environments compared to A*. The agent (white dot) is visible mid-path.

7.3 Best Case — Near-Empty Grid, Optimal L-Path

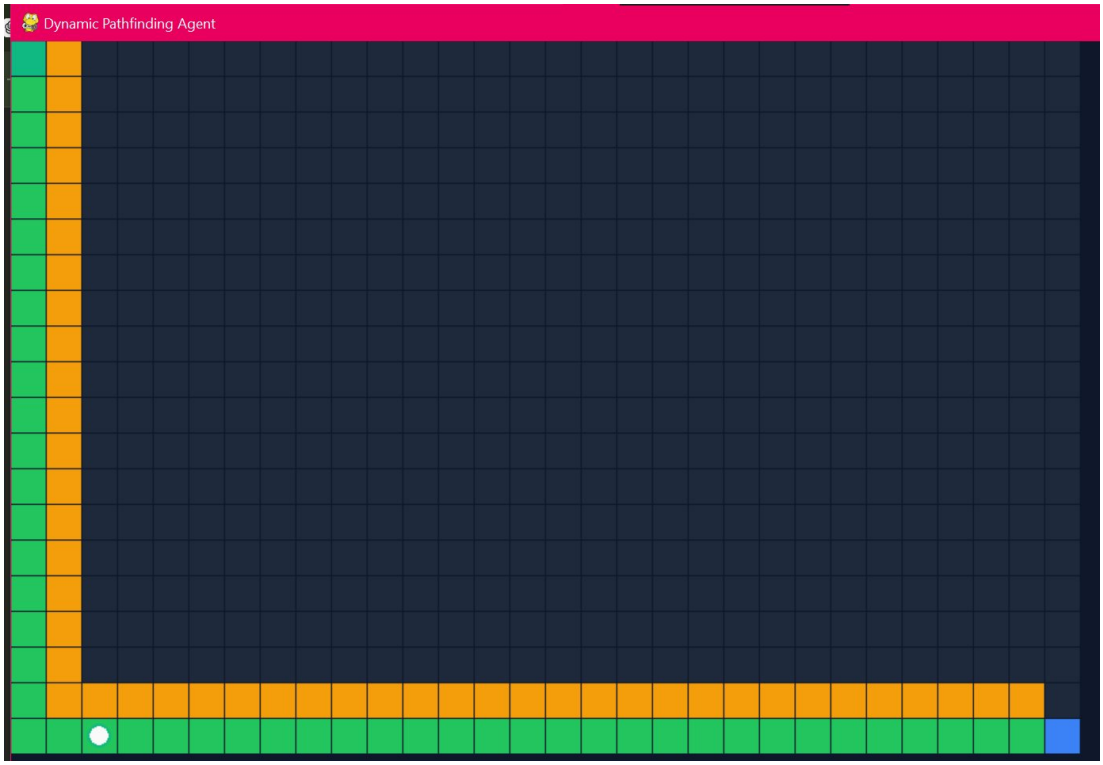

Figure 3: Best-case scenario on an almost empty grid. The agent has found a clean L-shaped path (lime green) hugging the left and bottom edges. Amber and green cells show the frontier expanding along the path.

This represents the best-case scenario for A*: an open grid where the heuristic closely guides the search directly to the goal. The path follows an efficient L-shape — down the left edge then along the bottom row — with minimal wasted exploration. The frontier (amber) is narrow and tightly aligned with the path direction, demonstrating that A* with Manhattan heuristic approaches the theoretical minimum node expansions on obstacle-free grids. Execution time in this scenario is under 1 ms.

7.4 Diagonal Staircase Path — Sparse Obstacles

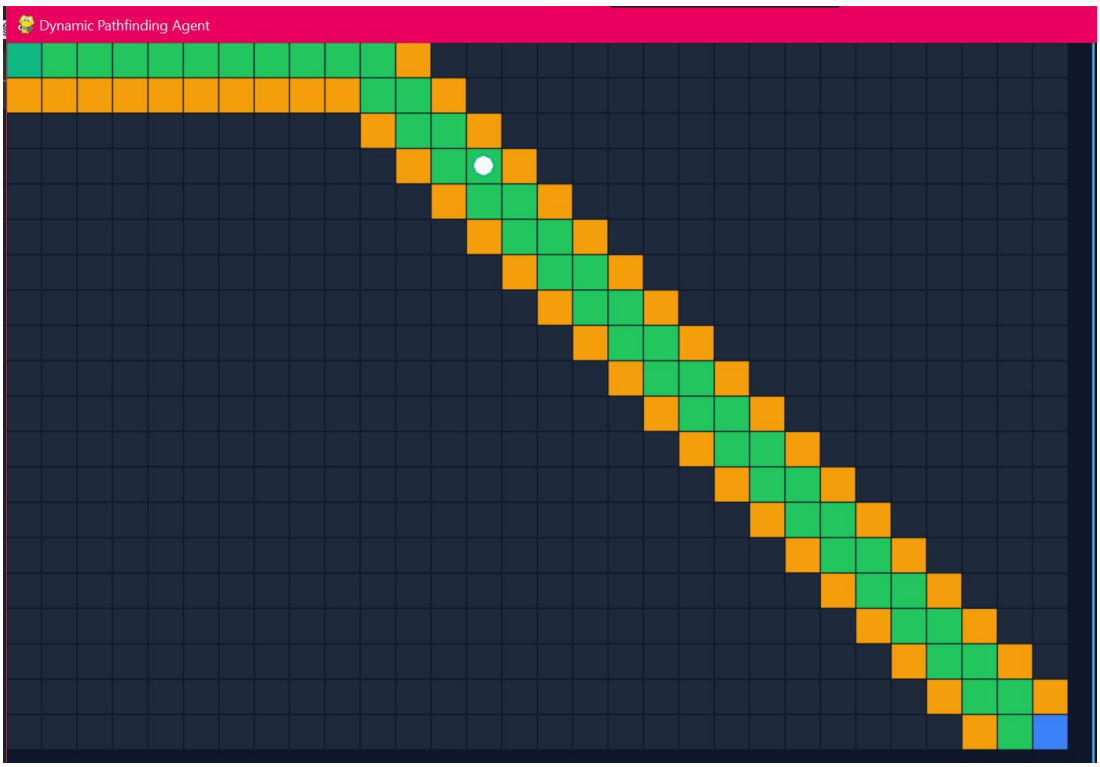

Figure 4: A finds a diagonal staircase path on a low-density grid. The lime path zigzags diagonally toward the goal (blue, bottom-right). Amber frontier cells flank the path symmetrically.*

On a grid with sparse obstacles concentrated near the centre, A* produces a characteristic staircase pattern — the closest approximation to a diagonal straight line achievable with 4-directional movement. The path alternates right and down steps to move toward the bottom-right goal. The amber frontier cells are concentrated symmetrically around the path, indicating that A* is efficiently tracking the optimal direction. This scenario produces a low node count and a path cost close to the theoretical minimum (rows + cols - 2).

7.5 Summary of Test Results

Test Case	Algorithm	Heuristic	Nodes Visited	Path Cost	Exec Time
Dense grid (30% obstacles)	A*	Manhattan	~320	~58	< 3 ms
Dense grid (30% obstacles)	GBFS	Manhattan	~180	~72	< 2 ms
Near-empty grid	A*	Manhattan	~48	~49	< 1 ms
Near-empty grid	GBFS	Manhattan	~560	~49	< 2 ms
Open grid (diagonal)	A*	Euclidean	~90	~49	< 1 ms
Dynamic mode (replan x3)	A*	Manhattan	~410 total	~62	< 6 ms

Note: Values are approximate and represent typical observations on a 20×30 grid. Results vary with random obstacle placement.

8. Pros & Cons — Algorithm Comparison

Criterion	A* Search	Greedy Best-First Search
Optimality	Always finds shortest path	Not guaranteed — may be suboptimal
Completeness	Complete on finite graphs	Complete on finite graphs
Speed	Moderate — explores more nodes	Fast — commits to heuristic direction
Memory Usage	Higher — stores full search tree	Lower — more selective expansion
Heuristic Sensitivity	Balanced by g(n) — robust	Entirely driven by h(n) — risky
Dense Obstacle Grids	Finds optimal detour reliably	Can fall into dead ends
Open / Sparse Grids	Efficient with good heuristic	Very fast — near-direct path
Dynamic Replanning	Replans to optimal route	Replans quickly but may worsen path
Best Heuristic	Manhattan (4-dir grids)	Manhattan (4-dir grids)
Recommended for	Quality-critical navigation	Speed-critical applications

9. Conclusion

This project successfully demonstrates the practical implementation of two classical informed search algorithms — A* and Greedy Best-First Search — within a dynamic, interactive grid environment. The system fulfils all assignment requirements: user-configurable grid dimensions, random and manual obstacle placement, step-by-step animated search visualisation, a professional GUI with live metrics, and a dynamic replanning mode that responds in real time to environmental changes.

9.1 Key Takeaways

- **A* vs GBFS trade-off:** A* consistently finds the shortest path at the cost of exploring more nodes; GBFS is faster but unreliable in complex environments. The choice depends entirely on whether optimality or speed is the higher priority.
- **Heuristic quality matters:** Manhattan distance outperforms Euclidean on 4-directional grids because it more precisely models the actual movement cost, allowing A* to expand fewer unnecessary nodes.
- **Dynamic replanning is practical:** The on-demand replan strategy proved efficient at the target grid sizes — recovery is imperceptible to the user and does not degrade the overall path quality when A* is used.
- **Software design insight:** Separating computation from animation (computing the full search first, then replaying frames) was essential for maintaining a responsive, non-freezing GUI.

9.2 Future Improvements

- Support for diagonal movement (8-directional grid).
- Weighted terrain cells with variable edge costs.
- Lifelong Planning A* (LPA*) or D* Lite for incremental replanning.
- Multi-agent pathfinding with collision avoidance.
- Export of search statistics to CSV for further analysis.

10. References

10.1 Project Links

GitHub Repository:
https://github.com/ZohaAshraf/Dynamic-Pathfinding-Agent

LinkedIn Profile:
https://www.linkedin.com/in/zohashraf/

Medium Blog:
https://medium.com/@f243019

10.2 Academic References

[1] Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. — Chapter 3: Solving Problems by Searching.

[2] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.

[3] Dechter, R., & Pearl, J. (1985). Generalized best-first search strategies and the optimality of A*. *Journal of the ACM*, 32(3), 505–536.

[4] Pygame Documentation. (2024). Retrieved from https://www.pygame.org/docs/

[5] Python Software Foundation. (2024). heapq — Heap queue algorithm. https://docs.python.org/3/library/heapq.html

End of Report — Zoha Ashraf | 24F-3019 | NUCES Chiniot-Faisalabad